

E-Commerce Order Management System :

Area	Technology
Language	Java 17
Framework	Spring Boot
API	REST
Service Discovery	Eureka
Gateway	Spring Cloud Gateway
Security	Spring Security + JWT
Database	PostgreSQL
Communication	REST + OpenFeign
Async Communication	Apache Kafka
Cache	Redis
Resilience	Resilience4j
Containerization	Docker + Docker Compose
Testing	JUnit 5 + Mockito + Testcontainers
Documentation	Swagger/OpenAPI
Monitoring	Actuator + Prometheus + Grafana
Logging	SLF4J + centralized logging
Build	Maven

2. Most Important Microservice Rule

Each service should have **its own database/schema**.

Don't create one common database like:

ecommerce_db

└─ users

└─ products

└─ orders

└─ payments

└─ inventory

Instead:

User Service → user_db

Product Service → product_db

Order Service → order_db

Payment Service → payment_db

Inventory Service → inventory_db

Notification → notification_db

This gives you **database-per-service**, which is an important microservices concept.

3. USER SERVICE

Responsible for:

- Registration
- Login
- User profile
- Address
- Roles
- JWT authentication

Database

users

id UUID PK

first_name

last_name

email UNIQUE

password

phone

status

role

created_at

updated_at

Example:

users

id	UUID
first_name	Rushikesh
last_name	Mind
email	rushi@gmail.com
password	encrypted
phone	9876543210
status	ACTIVE
role	CUSTOMER
created_at	

updated_at

Address table

addresses

id UUID PK

user_id UUID

address_line1

address_line2

city

state

country

pincode

address_type

is_default

created_at

Relationship:

User

|

└───< Address

One user can have multiple addresses.

4. PRODUCT SERVICE

Responsible for:

- Product creation
- Product update
- Product search
- Categories
- Product pricing
- Product status

Category

categories

id UUID PK

name

description

status

created_at

updated_at

Product

products

id UUID PK

category_id

name

description

sku UNIQUE

price

currency

status

brand

created_at

updated_at

Example:

Product

id = UUID

name = iPhone 17

sku = IP17-256-BLK

price = 79999

currency = INR

status = ACTIVE

Product Image

product_images

id UUID PK

product_id UUID

image_url

is_primary

created_at

Relationship:

Category

|

└──< Product

|

└──< ProductImage

5. INVENTORY SERVICE

This is separate from Product Service.

Product Service says:

"What is this product?"

Inventory Service says:

"How many are available?"

Inventory

inventory

id UUID PK

product_id UUID UNIQUE

available_quantity

reserved_quantity

sold_quantity

reorder_level

version

updated_at

Example:

Product: iPhone 17

available_quantity = 50

reserved_quantity = 5

sold_quantity = 100

Inventory Reservation

Very important for real-world order processing.

inventory_reservations

id UUID PK

order_id UUID

product_id UUID

quantity

status

reserved_at

released_at

Possible status:

RESERVED

RELEASED

CONFIRMED

EXPIRED

6. ORDER SERVICE

This is the **central business service**.

It manages:

- Cart/order creation
- Order items
- Order status
- Order cancellation
- Order history

Orders

orders

id UUID PK

user_id UUID

order_number UNIQUE

status

total_amount

currency

shipping_address

billing_address

created_at

updated_at

Example:

Order

id UUID

order_number ORD-2026-00001

user_id UUID

status CREATED

total_amount 85000

currency INR

Order Items

order_items

id UUID PK

order_id UUID

product_id UUID

product_name

sku

quantity

unit_price

total_price

created_at

Notice something important:

We store:

product_name

unit_price

inside order_items.

Why?

Suppose today:

iPhone = ₹80,000

Customer orders it.

Tomorrow:

iPhone = ₹85,000

The old order should still show:

₹80,000

Therefore, an order should preserve the **price snapshot**.

7. PAYMENT SERVICE

Responsible for:

- Payment initiation

- Payment processing
- Payment status
- Refund
- Transaction history

Payments

payments

id UUID PK

order_id UUID

user_id UUID

payment_reference UNIQUE

amount

currency

payment_method

status

created_at

updated_at

Payment methods:

CARD

UPI

NET_BANKING

WALLET

COD

Payment status:

INITIATED

PROCESSING

SUCCESS

FAILED

REFUNDED

Payment Transaction

payment_transactions

id UUID PK

payment_id UUID

transaction_reference

gateway

gateway_transaction_id

amount

status

failure_reason

created_at

This is useful when an external payment gateway is involved.

8. NOTIFICATION SERVICE

Responsible for:

- Email
- SMS
- Push notifications

Notification

notifications

id UUID PK

user_id UUID

order_id UUID

type

channel

recipient

subject

message

status

retry_count

created_at

sent_at

Type:

ORDER_CREATED

PAYMENT_SUCCESS

PAYMENT_FAILED

ORDER_CONFIRMED

ORDER_CANCELLED

Channel:

EMAIL

SMS

PUSH

Status:

PENDING

SENT

FAILED

9. COMPLETE DATABASE STRUCTURE

Your overall system becomes:

USER DB

|

|— users

|— addresses

PRODUCT DB

|

└— categories
└— products
└ product_images

INVENTORY DB

|
└— inventory
└ inventory_reservations

ORDER DB

|
└— orders
└ order_items

PAYMENT DB

|
└— payments
└ payment_transactions

NOTIFICATION DB

|

└─ notifications

10. Important Microservices Concept

Don't create database foreign keys between services.

For example:

Order DB

orders

user_id

user_id is just a UUID/reference.

Don't do:

```
FOREIGN KEY(user_id)
```

```
REFERENCES user_db.users(id)
```

because Order Service must not directly depend on User Service's database.

Instead:

Order Service

|

| user_id



User Service

through REST/API if information is required.

11. API GATEWAY

Client should normally not call every service directly.

Instead:

React

|



API Gateway

|

└─ /api/users/**

└─ /api/products/**

└─ /api/orders/**

└─ /api/payments/**

└─ /api/inventory/**

Gateway responsibilities:

- Routing
- JWT validation
- Authentication
- CORS
- Rate limiting

- Logging
 - Request filtering
-

12. EUREKA SERVER

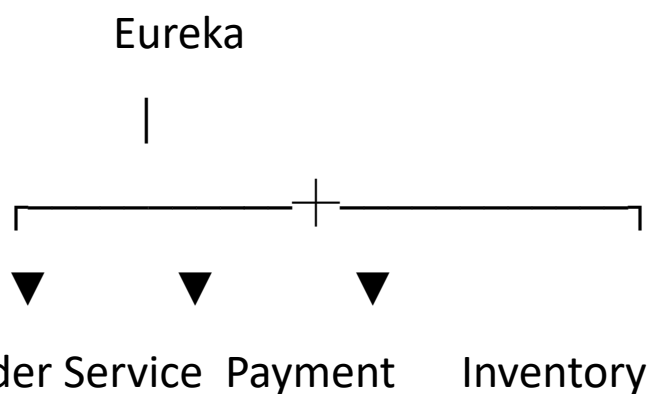
Eureka handles service discovery.

Without Eureka:

Order Service



With Eureka:



Order Service can discover:

PAYMENT-SERVICE

INVENTORY-SERVICE

instead of hardcoding IP/ports.

13. OPENFEIGN

For synchronous communication.

Example:

Order Service

|

| Feign



Inventory Service

Example:

```
@FeignClient(name = "INVENTORY-SERVICE")
```

```
public interface InventoryClient {
```

```
    @PostMapping("/api/inventory/reserve")
```

```
    InventoryResponse reserve(
```

```
        @RequestBody InventoryRequest request);
```

```
}
```

14. KAFKA

Kafka should handle asynchronous events.

For example:

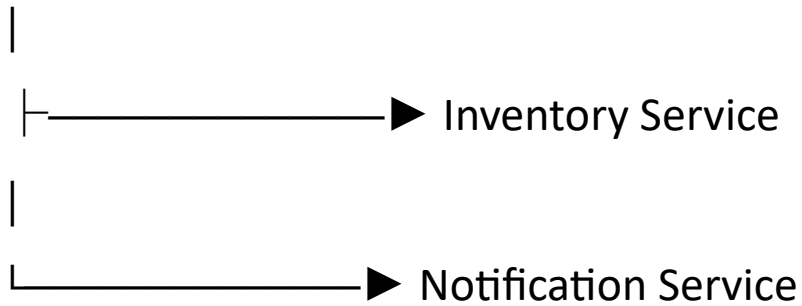
Order Service

|

| OrderCreatedEvent



Kafka



Possible topics:

order-events

payment-events

inventory-events

notification-events

Events:

OrderCreated

PaymentSuccessful

PaymentFailed

InventoryReserved

InventoryReleased

OrderCancelled

15. COMPLETE ORDER WORKFLOW

This is the most important part of your project.

Suppose customer buys:

iPhone

Quantity = 1

Price = ₹80,000

Step 1 — Login

Customer

|



API Gateway

|



User Service

|



Validate credentials

|



Generate JWT

Response:

JWT Token

16. Step 2 — Browse Products

Customer

|



API Gateway

|



Product Service

|



PostgreSQL

Response:

```
{  
  "id": "123",  
  "name": "iPhone",  
  "price": 80000,  
  "currency": "INR"  
}
```

17. Step 3 — Create Order

Customer clicks:

BUY NOW

Request:

POST /api/orders

```
{  
  "productId": "123",  
  "quantity": 1  
}
```

Flow:

Customer



API Gateway



Order Service



Validate user



Get product details



Calculate total



Create Order

Order:

CREATED

18. Step 4 — Reserve Inventory

Order Service asks Inventory Service:

Reserve iPhone

Quantity = 1

Inventory:

Available = 10

After reservation:

Available = 9

Reserved = 1

Reservation:

RESERVED

19. Step 5 — Payment

Now:

Order Service

|



Payment Service

|



Payment Gateway

Payment:

INITIATED



PROCESSING



SUCCESS

20. Step 6 — Confirm Order

After successful payment:

Payment Service



| PaymentSuccessful



Kafka



Order Service



Order CONFIRMED

Inventory becomes:

RESERVED → CONFIRMED

21. Step 7 — Notification

Kafka event:

OrderConfirmed

Notification Service consumes it.

Kafka

|



Notification Service

|

├— Email

├— SMS

└— Push

Customer receives:

Your order ORD-2026-00001 has been confirmed.

22. Complete Workflow

CUSTOMER

|

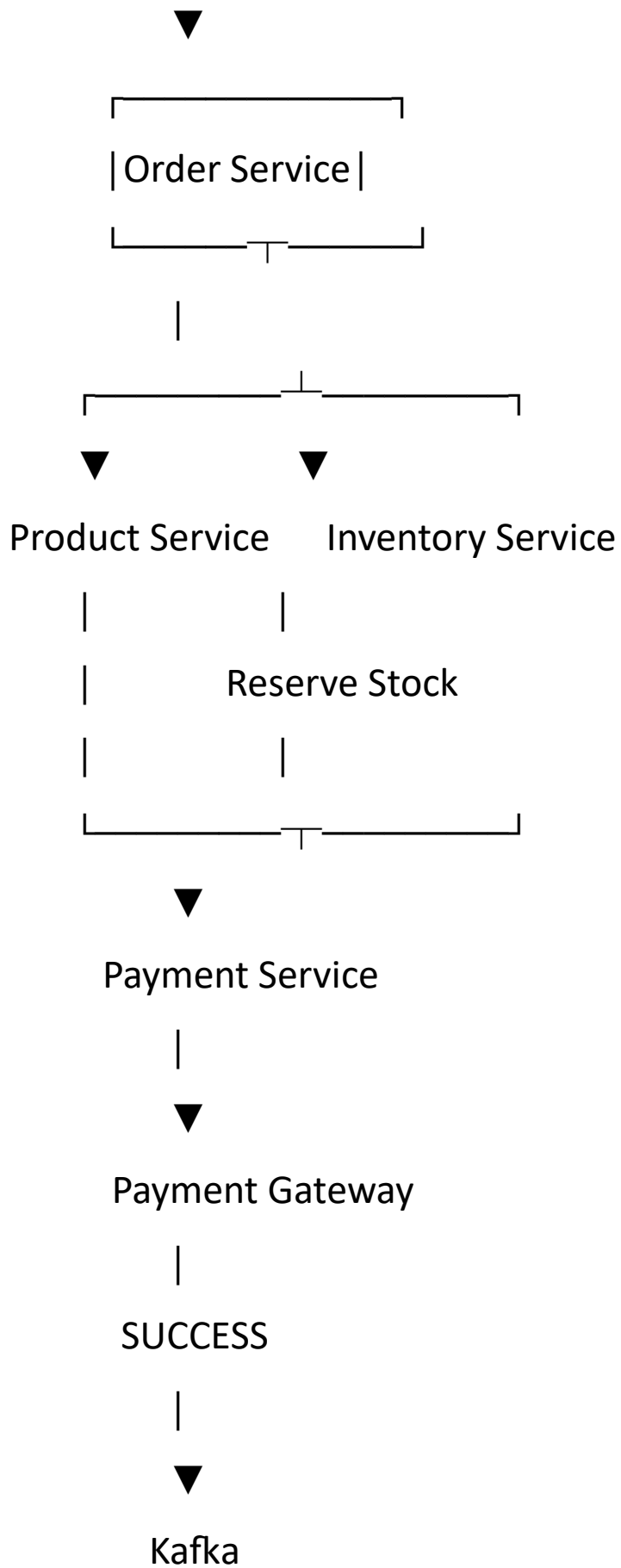


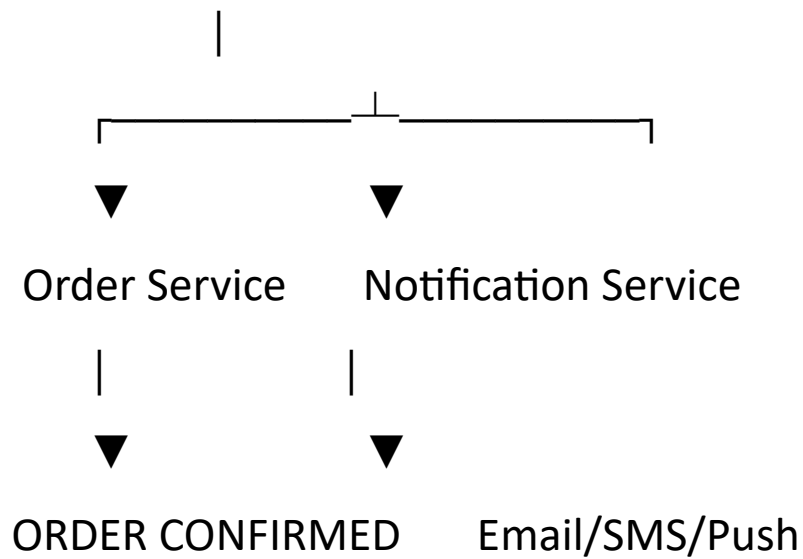
┌──────────────────┐

| API Gateway |

└──────────┴────────┘

|





23. What if Payment Fails?

This is where the project becomes interesting.

Suppose:

Inventory reserved



Payment failed

We cannot leave inventory reserved forever.

So:

Payment Failed



Kafka



Order Service



Cancel Order



Release Inventory

Inventory:

Reserved → Released

Available stock increases again.

24. Saga Pattern

This project is perfect for learning **Saga Pattern**.

Because we have:

Order

Inventory

Payment

and they each have their own database.

We should **not** use one giant database transaction.

Instead:

Create Order



Reserve Inventory



Process Payment



Confirm Order

If something fails:

Payment Failed



Release Inventory



Cancel Order

This is a **distributed transaction** handled using Saga.

25. Saga — Choreography

Kafka-based choreography:

OrderCreated



Kafka



Inventory Service



InventoryReserved



Kafka



Payment Service

↓
PaymentSuccessful

↓
Kafka

↓
Order Service

↓
OrderConfirmed

Failure:

PaymentFailed

↓
Kafka

↓
Inventory Service

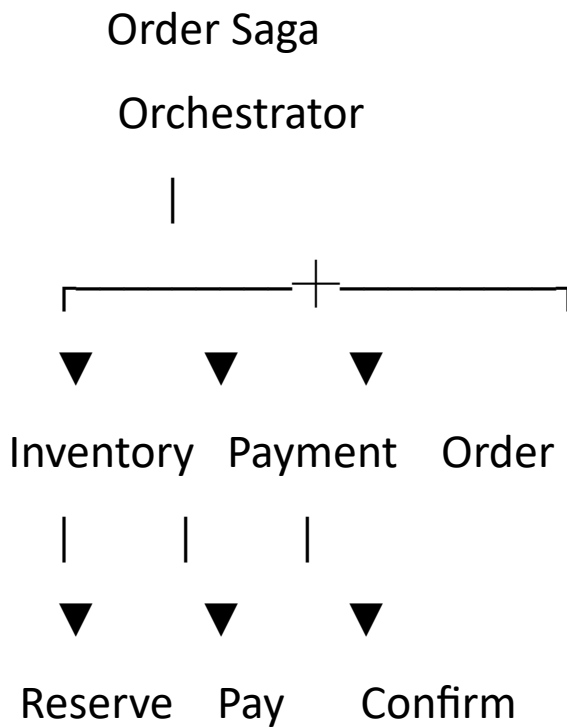
↓
ReleaseInventory

↓
OrderCancelled

This is a very good interview topic.

26. Saga — Orchestration

Another approach is an **Order Saga Orchestrator**:



For your first implementation, I recommend learning:

Kafka + choreography first, then understand orchestration.

27. Redis

Redis can be used for:

Product caching

Client



Product Service



Redis



Product DB

Instead of querying PostgreSQL every time:

GET /products/123

first check:

Redis

If available:

CACHE HIT

Otherwise:

PostgreSQL



Redis



Response

28. Resilience4j

Imagine Payment Service is down.

Without resilience:

Order → Payment



Timeout



Error

Use:

Circuit Breaker

Retry

Timeout

Fallback

Example:

Order Service

|



Payment Service

|

X

unavailable

|



Circuit Breaker

|



Fallback

29. Idempotency

Very important for Payment and Order.

Suppose customer clicks:

PAY

twice because the network is slow.

Without idempotency:

Payment 1 → SUCCESS

Payment 2 → SUCCESS

Customer gets charged twice.

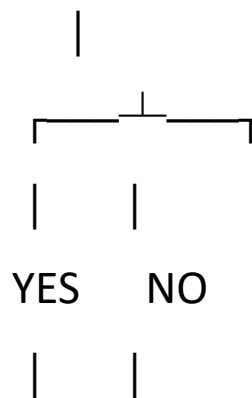
Instead:

Idempotency-Key:

PAY-ORD-2026-00001

Payment Service checks:

Already processed?



```
return process
```

existing

result

30. Security

Use:

Spring Security

+

JWT

+

Role Based Access Control

Roles:

CUSTOMER

ADMIN

PRODUCT_MANAGER

INVENTORY_MANAGER

Example:

CUSTOMER

└─ View products

└─ Create order

└─ View own orders

└─ Make payment

ADMIN

└─ Manage users

└─ Manage products

└─ View orders

31. Project Package Structure

Each service should follow a clean structure.

Example Order Service:

order-service

```
|
|
|  └─ src/main/java
|    └─ com.ecommerce.order
|
|
|      └─ controller
|        └─ OrderController.java
|
|      └─ service
|        └─ OrderService.java
|        └─ OrderServiceImpl.java
|
|      └─ repository
|        └─ OrderRepository.java
|
|      └─ entity
|        └─ Order.java
|        └─ OrderItem.java
```

- | |
- | |└─ dto
- | | |└─ OrderRequest.java
- | | |└─ OrderResponse.java
- | |
- | |└─ client
- | | |└─ ProductClient.java
- | | |└─ InventoryClient.java
- | | |└─ PaymentClient.java
- | |
- | |└─ kafka
- | | |└─ OrderProducer.java
- | | |└─ OrderConsumer.java
- | |
- | |└─ exception
- | | |└─ OrderNotFoundException.java
- | | |└─ GlobalExceptionHandler.java
- | |
- | |└─ config
- | | |└─ KafkaConfig.java
- | | |└─ SecurityConfig.java

```
|
|
|   └─ OrderApplication.java
|
└─ pom.xml
```

32. Complete Repository Structure

I would keep your project like this:

ecommerce-microservices/

```
|
|└─ api-gateway/
|
|└─ eureka-server/
|
|└─ config-server/
|
|└─ user-service/
|
|└─ product-service/
|
|└─ inventory-service/
|
```

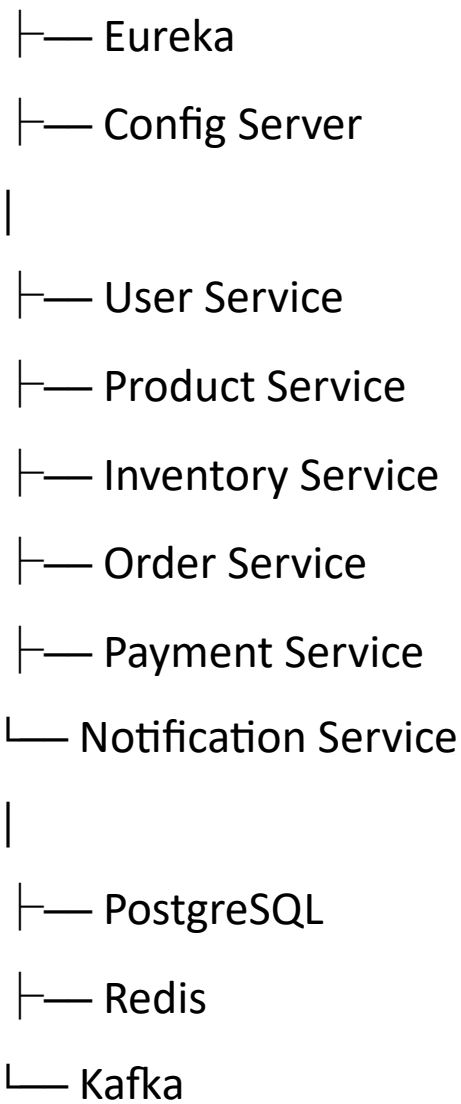
- └─ order-service/
- |
- └─ payment-service/
- |
- └─ notification-service/
- |
- └─ docker-compose.yml
- |
- └─ README.md
- |
- └─ docs/
 - └─ architecture/
 - └─ database/
 - └─ api/
 - └─ workflows/

33. Docker Architecture

Eventually:

Docker Compose

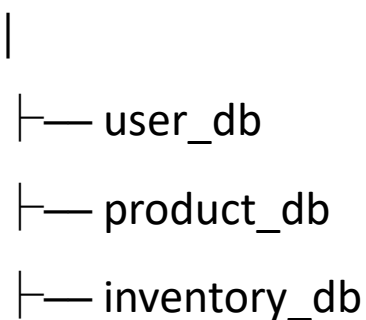
- |
- └─ API Gateway



You can initially use one PostgreSQL container with **separate databases**, then move toward fully separate DB instances if needed.

Example:

PostgreSQL



└─ order_db

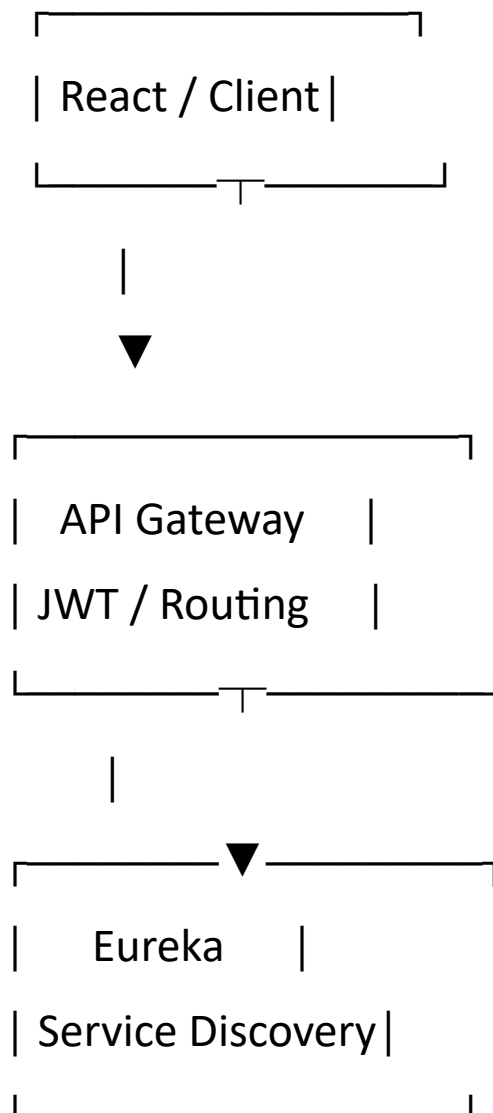
└─ payment_db

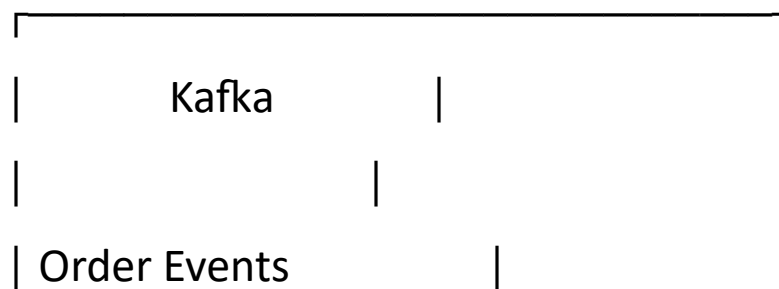
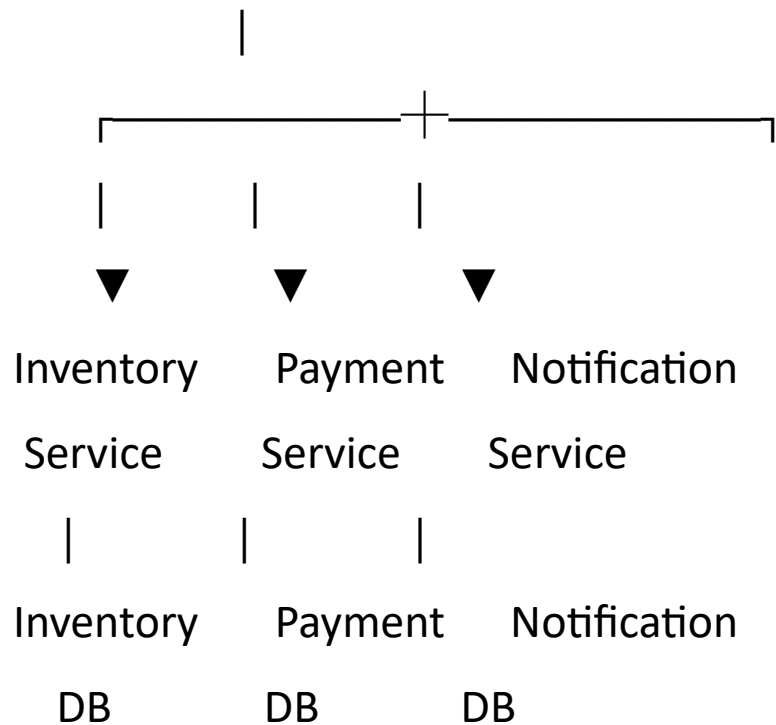
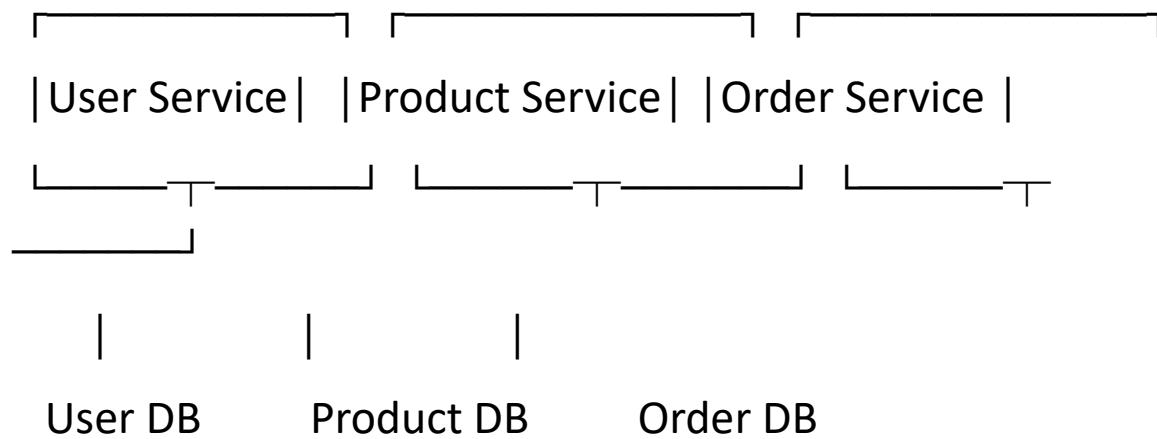
└─ notification_db

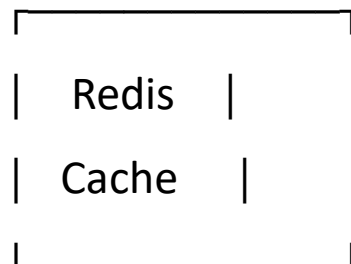
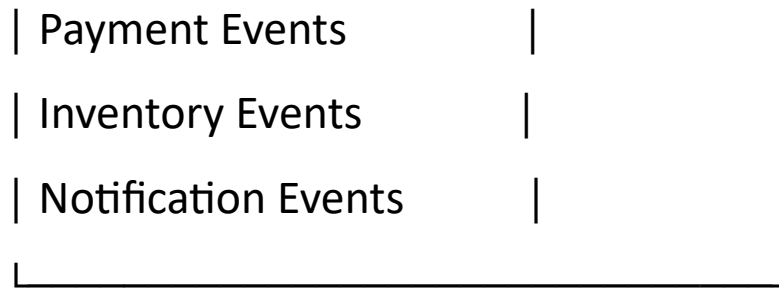
Logically, each service owns its own database.

34. Final Architecture

Your final project can look like this:







Learning/Implementation Order

Don't try to build everything at once. Build it in this order:

Phase 1 — Foundation

1. Create Git repository
2. Create Eureka Server
3. Create API Gateway
4. Create Config Server
5. Create all Spring Boot services

Phase 2 — Core CRUD

6. User Service
7. Product Service
8. Inventory Service
9. Order Service

- 10. Payment Service
- 11. Notification Service

Phase 3 — Communication

- 12. OpenFeign
- 13. REST communication
- 14. Kafka
- 15. Kafka producers/consumers
- 16. Event-driven order workflow

Phase 4 — Real-world problems

- 17. Saga Pattern
- 18. Idempotency
- 19. Retry
- 20. Circuit Breaker
- 21. Distributed transaction
- 22. Inventory reservation
- 23. Payment failure/rollback

Phase 5 — Production

- 24. JWT + RBAC
- 25. Redis
- 26. Docker
- 27. Docker Compose
- 28. Actuator
- 29. Prometheus/Grafana
- 30. Distributed tracing
- 31. Centralized logging
- 32. Testcontainers
- 33. CI/CD